Project Overview

What OxCidation Studies

OxCidation studies whether a language model can translate a complete accepted C program into Rust without changing what the program does. The current study compares translation prompts: the model remains fixed, while the C program and the prompt vary.

The project evaluates more than compilation. A useful translation should:

- produce the same answers as the C program;
- avoid crashes, panics, and unexpected behavior;
- remain within the problem's time and memory constraints.

What One Result Represents

The basic result is one combination:

```
accepted C program + translation prompt + fixed model
```

For that combination, OxCidation keeps both the first Rust translation and the final Rust version after any allowed repairs. This distinction lets the study answer two different questions:

1. How effective is the translation prompt by itself?
2. How effective is the complete translation-and-repair pipeline?

The Two Evaluation Contexts

The pipeline deliberately separates evidence that may guide repair from evidence used only for final observation.

Visible tests

The model proposes test inputs after reading only the C source. OxCidation runs each candidate twice against C, rejects unusable or unstable inputs, and asks the Validator to review every candidate before any translation begins. Approved cases are preserved; rejected cases may receive one replacement round. The resulting suite is frozen and shared by every prompt. Later C/Rust differences can be analyzed by the Validator and may lead to repair.

External Judge

A separate test suite evaluates C and Rust under configured time and memory limits. Its result is recorded for research analysis but is never sent back to the Translator. This prevents the pipeline from adapting the Rust code to the same evidence later used to evaluate it.

Experiment Lifecycle

The complete research process has three stages:

```
1. Prepare population
   Extract accepted C programs and select reproducible samples.
```

```
2. Select a prompt
   Run every candidate prompt on the same prompt-selection programs.

3. Evaluate the selected prompt
   Freeze the chosen prompt and run it on a separate, untouched set of problems.
```

The second and third stages must use different problems. Otherwise, the final performance estimate would favor a prompt chosen specifically because it did well on those same examples.

What Is Already Built

- Complete accepted-C extraction from Project CodeNet.
- Selection manifests with stable seeds.
- Multi-prompt scheduling with one model per experiment.
- Reviewed visible-test generation and differential C/Rust execution.
- Translation and bounded repair flow.
- Initial and final Judge evaluation.
- Detailed artifacts for programs, attempts, reports, and agent interactions.
- Resumability and isolated experiment directories.
- CSV, JSON, summary, and Excel-compatible reports.

What Is Not Ready Yet

The pipeline works, but the definitive study cannot start until the team fixes or decides:

- the model and exact configuration;
- sample sizes and programs selected per problem;
- the metric used to choose a prompt;
- the prompt-selection and final-evaluation split;
- the source and coverage of external Judge cases.

Where the Implementation Lives

Readers do not need these files to understand the experiment, but this map is useful when following development:

Area	Main files
Benchmark scheduling and resume	`src/benchmark.py`, `src/benchmark_experiment.py`
Agent flow	`src/orchestrator.py`, `src/agents.py`, `src/validator.py`
LLM and compiler tools	`src/server.py`
Generated visible tests	`src/visible_test_preparation.py`, `src/visible_testing.py`
External Judge execution	`src/judge_comparison.py`, `src/judge_execution.py`
Reports and workbook	`src/benchmark_reporting.py`, `src/review_workbook.py`
CodeNet preparation	`extract_accepted_codenet.py`, `prepare_benchmark_population.py`, `prepare_aoj_problem_mapping.py`

AOJ suite preparation	`aoj_api.py`, `prepare_aoj_system_tests.py`

Claims the Project Must Avoid

- Passing generated visible tests is not proof of correctness.
- CodeNet statement samples are not a comprehensive Judge suite.
- Multiple programs from one problem are related observations, not independent problems.
- A Judge failure describes evaluation evidence; it is not repair feedback.
- Performance measured on the prompt-selection set is not the final performance claim.